

1 RideFlow Deployment, Stripe, and M-Pesa Daraja Guide

Audience: RideFlow owner/operator moving the application to a self-managed server.

Repository: <https://github.com/raiven427/ride-flow>

Currency in the current Kenya fare model: KES/KSh

I am an AI, not a licensed financial, legal, tax, or payments professional. Before accepting real fares, have a qualified Kenyan payments/legal/tax professional review marketplace obligations, driver contracts, consumer rules, data protection, tax treatment, refunds, and payout structure.

1.1 What is ready

The repository now contains environment-driven production scaffolding. The server has a health endpoint at `/healthz`, a protected admin payment-readiness status procedure, Stripe webhook signature verification at `/api/webhooks/stripe`, and a Daraja callback entry point at `/api/webhooks/daraja`. Stripe Connect payment-intent and Daraja STK Push helpers are server-side and disabled until their environment flags are explicitly enabled.

The existing database schema, fare rules, 5% commission ledger, admin controls, storage adapter, and tests remain in place. No API keys are included in source control. The application is not automatically production-ready merely because the keys are entered: provider onboarding, webhook configuration, reconciliation, driver payouts, testing, and operational controls still have to be completed.

1.2 1. Choose your server

You need a Linux server or managed Node.js host that supports Node.js LTS, pnpm or npm, HTTPS, environment secrets, outbound HTTPS requests, and a persistent database connection. A development PC is not a suitable production server unless it is deliberately secured, reachable, monitored, backed up, and kept running.

A practical first deployment uses a managed Node host or a small Linux VPS with a reverse proxy. Use one hostname for the web application, such as `https://rideflow.example.com`, and keep the database and object storage private whenever the provider allows it.

The server must expose HTTPS because OAuth callbacks, Stripe webhooks, and Daraja callbacks should not use public HTTP in production. Set the application process to restart on failure and add a health check for `/healthz`.

1.3 2. Clone and build RideFlow

On the server:

```
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install
pnpm check
pnpm test
```

```
pnpm build
NODE_ENV=production pnpm start
```

The application listens on `PORT`, defaulting to 3000. Put Nginx, Caddy, or your hosting provider's HTTPS proxy in front of it. Forward normal HTTP requests and WebSocket traffic if realtime features are later enabled.

1.4 3. Create and connect the database

Provision a MySQL- or TiDB-compatible database and a dedicated application account. Do not use the root database user from the application. Set:

```
DATABASE_URL=mysql://rideflow_app:strong-password@db-host:3306/rideflow
```

Generate and review migrations from `drizzle/schema.ts`:

```
pnpm drizzle-kit generate
```

Apply migrations using your approved migration process. Never run destructive SQL against the existing project database as part of this setup. A new database receives the schema; existing data requires a separate backup, restore, verification, and cutover plan.

Before launch, verify users, profiles, fare rules, quotes, ledger entries, admin settings, and uploaded-file metadata. Take encrypted backups and practice restoring them to a temporary database.

1.5 4. Configure private file storage

Create a private S3-compatible bucket and service account. The application uses the following values:

```
S3_BUCKET=rideflow-private
S3_REGION=auto
S3_ENDPOINT=https://your-storage-endpoint.example
S3_ACCESS_KEY_ID=replace-me
S3_SECRET_ACCESS_KEY=replace-me
S3_FORCE_PATH_STYLE=false
S3_SIGNED_URL_TTL_SECONDS=900
```

Store profile photos, driver licenses, insurance documents, vehicle documents, and lost-item attachments in private storage. Keep only metadata and storage keys in the database. Test upload limits, MIME validation, authorization, signed URL expiry, and recovery after a storage outage.

1.6 5. Configure authentication and notifications

Choose an OIDC provider and register this callback:

```
https://your-domain.example/api/oauth/callback
```

Configure:

```
OAUTH_ISSUER_URL=https://identity.example
OAUTH_TOKEN_URL=https://identity.example/oauth/token
OAUTH_USERINFO_URL=https://identity.example/oauth/userinfo
OAUTH_CLIENT_ID=replace-me
OAUTH_CLIENT_SECRET=replace-me
VITE_OAUTH_AUTHORIZE_URL=https://identity.example/oauth/authorize
VITE_OAUTH_CLIENT_ID=replace-me
VITE_OAUTH_SCOPE=openid profile email
JWT_SECRET=generate-a-long-random-value
```

Keep the current admin email in the database-backed admin settings. Use `RIDEFLOW_ADMIN_EMAIL=njengastephen112@gmail.com` only as an initialization/operations value; do not hardcode ownership rules into frontend code.

Configure a real notification provider or webhook:

```
NOTIFICATION_PROVIDER_URL=https://your-notification-webhook.example/rideflow
NOTIFICATION_PROVIDER_TOKEN=replace-me
```

Test first signup, duplicate login, notification failure, notification retry, admin transfer, and unauthorized admin actions.

1.7 6. Stripe Connect setup

Stripe Connect is the marketplace-oriented Stripe product for platforms that collect from customers and pay service providers. Follow the official Connect marketplace documentation before selecting the charge and payout model.^[1]

Create and verify the Stripe platform account. Enable Connect, configure driver/connected-account onboarding, and decide whether drivers use Express, Standard, or another supported account type. Confirm who is merchant of record, how disputes and refunds affect drivers, and when driver payouts are released.

Place these values only in the server environment:

```
PAYMENTS_STRIPE_ENABLED=false
STRIPE_SECRET_KEY=sk_test_your_key
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_your_key
STRIPE_WEBHOOK_SECRET=whsec_your_endpoint_secret
STRIPE_CONNECT_CLIENT_ID=ca_your_connect_client_id
```

Set `PAYMENTS_STRIPE_ENABLED=true` only after sandbox tests pass. The browser may receive a publishable key, but never expose `STRIPE_SECRET_KEY` or `STRIPE_WEBHOOK_SECRET`.

Register this production webhook:

```
https://your-domain.example/api/webhooks/stripe
```

Stripe requires webhook signature verification using the raw request body and `Stripe-Signature` header. The RideFlow route performs the signature check and returns a 2xx response for verified events. Add

event processing for payment success, payment failure, refunds, disputes, connected-account updates, and payout failures before real launch.[2] [3]

The current server helper creates a payment intent in KSh and can attach an application fee and connected-account destination. Connect onboarding and final payout/reconciliation must still be completed for your business model. Do not treat the visible 5% ledger line as proof that money has been transferred.

1.8 7. M-Pesa Daraja setup

Create an account in Safaricom's official Daraja portal, create a sandbox app, and enable the products required for your flow. Daraja 3.0 provides the portal, sandbox testing, and access to Safaricom/M-PESA APIs.[4]

For the initial customer payment flow, RideFlow is prepared for STK Push. You will need the correct business shortcode, passkey, callback URL, and approved production credentials. For driver disbursements, investigate the appropriate B2C or supported payout product separately; STK Push alone does not pay drivers.

Configure these server values:

```
PAYMENTS_DARAJA_ENABLED=false
DARAJA_ENVIRONMENT=sandbox
DARAJA_CONSUMER_KEY=replace-me
DARAJA_CONSUMER_SECRET=replace-me
DARAJA_SHORTCODE=replace-me
DARAJA_PASSKEY=replace-me
DARAJA_INITIATOR_NAME=replace-me
DARAJA_SECURITY_CREDENTIAL=replace-me
DARAJA_CALLBACK_BASE_URL=https://your-domain.example/api/webhooks/daraja
```

The server requests a Daraja access token, creates the STK Push password from shortcode, passkey, and timestamp, and sends the request server-side. It validates Kenyan phone numbers in 254XXXXXXXX form. Do not put consumer keys, consumer secrets, passkeys, or security credentials in React code.

Configure callbacks that can be reached over HTTPS. Persist the provider checkout/request reference and callback result against the RideFlow payment and fare ledger. Handle timeout, cancellation, duplicate callback, reversal, and payment-without-ride cases. Enable production only after sandbox tests and provider approval pass.

1.9 8. Fare, commission, and payout rules

RideFlow calculates the quote on the server using database-backed Nairobi fare rules. The current commission basis is 5% or 500 basis points. For a completed ride, the system should record the rider charge, driver earnings, platform commission, provider transaction reference, currency, refund amount, and payout status.

A safe money sequence is: create an expiring quote; authorize or request payment; create the ride; complete the ride; calculate the final fare on the server; record immutable ledger events; reconcile the provider event; then release the driver payout. Never trust a fare or commission number sent by the browser.

Before launch, decide whether the 5% applies to the whole fare or a defined eligible subtotal, who pays provider fees, how tips are treated, and how refunds and cancellations affect the driver/platform split. Document this in customer and driver terms.

1.10 9. Production environment template

Use `docs/deployment-environment.template` as the variable checklist. Keep actual values in the hosting provider's secret manager. Set both payment flags to false until the corresponding sandbox integration is verified. Use separate sandbox and production credentials, callback URLs, and webhook secrets.

Never commit `.env`, database dumps, private keys, payment credentials, identity documents, production logs, or webhook secrets. Rotate any secret that was pasted into a chat, issue, screenshot, or public repository.

1.11 10. Launch checklist

Area	Required before real rides
Server	HTTPS, process restart, health check, logs, domain, firewall
Database	reviewed migrations, least-privilege account, backups, restore test
Storage	private bucket, signed URLs, access tests, retention policy
Auth	OIDC callback, session secret, MFA for admins, transfer test
Stripe	verified platform, Connect onboarding, webhook signature checks, refunds/disputes, payout reconciliation
Daraja	sandbox tests, production approval, HTTPS callbacks, STK and reversal handling
Ride operations	dispatch, live location, chat, cancellation, support, emergency workflow
Finance	ledger reconciliation, driver statements, tax/accounting process, payout failures
Trust and safety	driver verification, incident response, women-only policy review, data retention
Security	rate limits, authorization tests, dependency updates, secret rotation

1.12 11. What you still need to build

The current code has payment configuration and provider entry points, but it does not magically complete a transport marketplace. You still need production maps and routing, driver dispatch, realtime location,

chat, scheduled-ride workers, customer payment-method UX, connected-driver onboarding, refunds, payout scheduling, fraud controls, support tools, tax reporting, and an incident-response process. Start in sandbox. Invite a small controlled pilot only after every payment callback is reconciled to the database and every test payment has a known final state. Do not turn on both payment flags for real customers until a payments professional and the providers' onboarding teams approve the structure.

1.13 References

[1]: <https://docs.stripe.com/connect> Stripe Connect marketplace documentation [2]: <https://docs.stripe.com/connect/webhooks> Stripe Connect webhooks documentation [3]: <https://docs.stripe.com/webhooks> Stripe webhook endpoint and signature documentation [4]: <https://developer.safaricom.co.ke/> Safaricom Daraja 3.0 developer portal